DevOps Pulse AI

Boletim inteligente de operações de desenvolvimento

Projeto acadêmico AI Factory — Etapa 1 / Unidade 2
Pipeline de automação: GitHub API → Python/FastAPI → Ollama → Edge TTS → SQLite → n8n → WhatsApp

Testes	Repo monitorado	Fonte de dados	Pipeline medido
21/21	**n8n-io/n8n**	**GitHub real**	**~83 s**

Itens cobertos (especificação da etapa)
ID 1.1 Diagnóstico · ID 2 Canvas (objetivos, entregáveis, papéis) · ID 2.1 Ambiente de automação · ID 2.2 Protótipo funcional · Aprendizagem-chave: input → process → output

Equipe de contexto: 5 devs (1 TL) + 1 PO · Persistência: SQLite · LLM local: Ollama · TTS: Edge (pt-BR)

Documento gerado em 17/09/2026 com evidências reais capturadas no ambiente local.

1. Diagnóstico do desafio real (ID 1.1)

Squad de 6 pessoas (5 desenvolvedores — sendo 1 Tech Lead — e 1 Product Owner) depende do GitHub para commits, pull requests, issues e CI. A informação fica espalhada em telas diferentes e a consolidação do boletim operacional é manual.

Processo manual observado

- Abrir o repositório e filtrar commits das últimas 24 h
- Listar PRs abertos e marcar os stale
- Checar issues e taxa de sucesso de Actions/CI
- Redigir resumo e enviar no chat/e-mail do time

Impacto e tempo

Atividade	Tempo	Quem
Consolidar boletim manual (UI GitHub → resumo → envio)	~25 min/dia (premissa)	TL
Supervisão da automação (dashboard + alertas)	~2 min/dia	TL / PO
Economia estimada	≈ 8,5 h/mês do TL	—

Oportunidade de automação

Ciclo formalizável: INPUT GitHub API → PROCESS métricas + alertas + LLM (Ollama) + áudio (Edge TTS) → OUTPUT JSON, SQLite com execution_id, n8n e WhatsApp.

Fontes espalhadas no GitHub (evidência do problema)

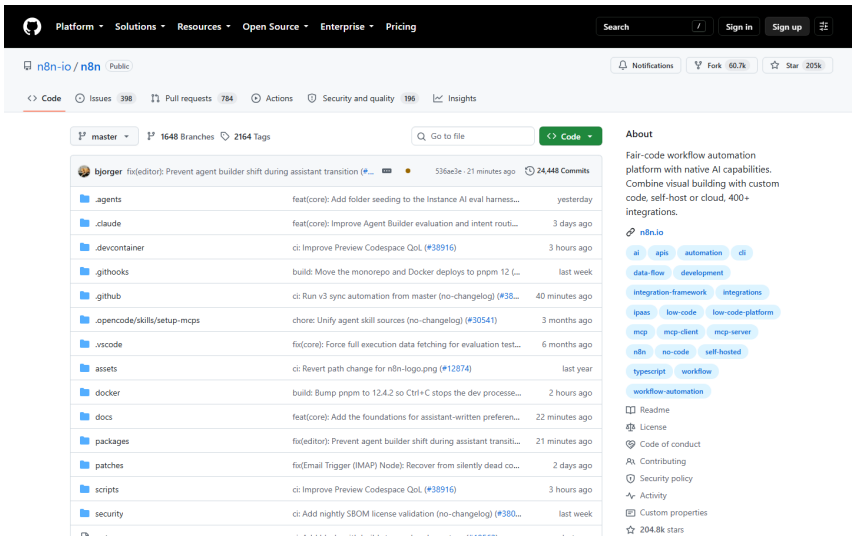


Fig. 1 — Repositório monitorado (n8n-io/n8n): commits/PRs/issues/CI em telas separadas.

2. Canvas do projeto — objetivos, entregáveis e papéis (ID 2)

Campo	Conteúdo
Problema	Informações de desenvolvimento espalhadas; consolidação manual atrasa decisões.
Público	Tech leads, PO e equipes de software.
Objetivo	Boletim operacional automatizado com indicadores, alertas e resumo executivo.
Proposta de valor	Menos esforço manual, rastreabilidade por execution_id, IA local, entrega em WhatsApp.
Entradas	GitHub API (commits, issues, PRs, workflow runs).
Processamento	Python: normalizar → métricas → alertas → Ollama → Edge TTS → SQLite.
Saídas	JSON /api/pulse, resumo em texto, áudio .wav, SQLite, notificações, dashboard.
Recursos / stack	Python 3.12, FastAPI, n8n, Ollama, Edge TTS, SQLite, Evolution API, Docker.
Riscos	API indisponível, rate limit, Ollama lento, TTS fora, alucinação, vazamento de token.
Mitigações	Retry, demo mode rotulado, fallback texto, .env gitignored, prompt restritivo, supervisão.
Entregáveis	Microsserviço, workflows n8n, 21 testes, stress test, dashboard, docs, evidências, PDF.

Papéis da equipe (6 pessoas)

Papel	Qtd	Responsabilidade
Tech Lead (1 dos 5 devs)	1	Dono do boletim, risco de CI/PR e revisão técnica.
Desenvolvedores	4	Implementação, testes, integrações e consumo do status.
Product Owner	1	Prioridade, requisitos, aceite e visão de valor/ROI.

Objetivos SMART (resumo)

- O1 Coleta de API pública · O2 Indicadores · O3 Resumo Ollama · O4 Áudio TTS · O5 Orquestração n8n
- O6 Persistência SQLite · O7 Multicanal · O8 Robustez (retries/logs/fallbacks) · O9 Performance · O10 Avaliação

3. Ambiente de automação (ID 2.1)

Chamadas HTTP em Python (FastAPI + curl), fluxos no n8n e Ollama/TTS executando localmente.

Saúde do sistema (componentes reais)



Fig. 2 — /health: GitHub, Ollama, TTS e n8n · demo_mode=false.

Infra local — Docker Desktop (containers no ar)

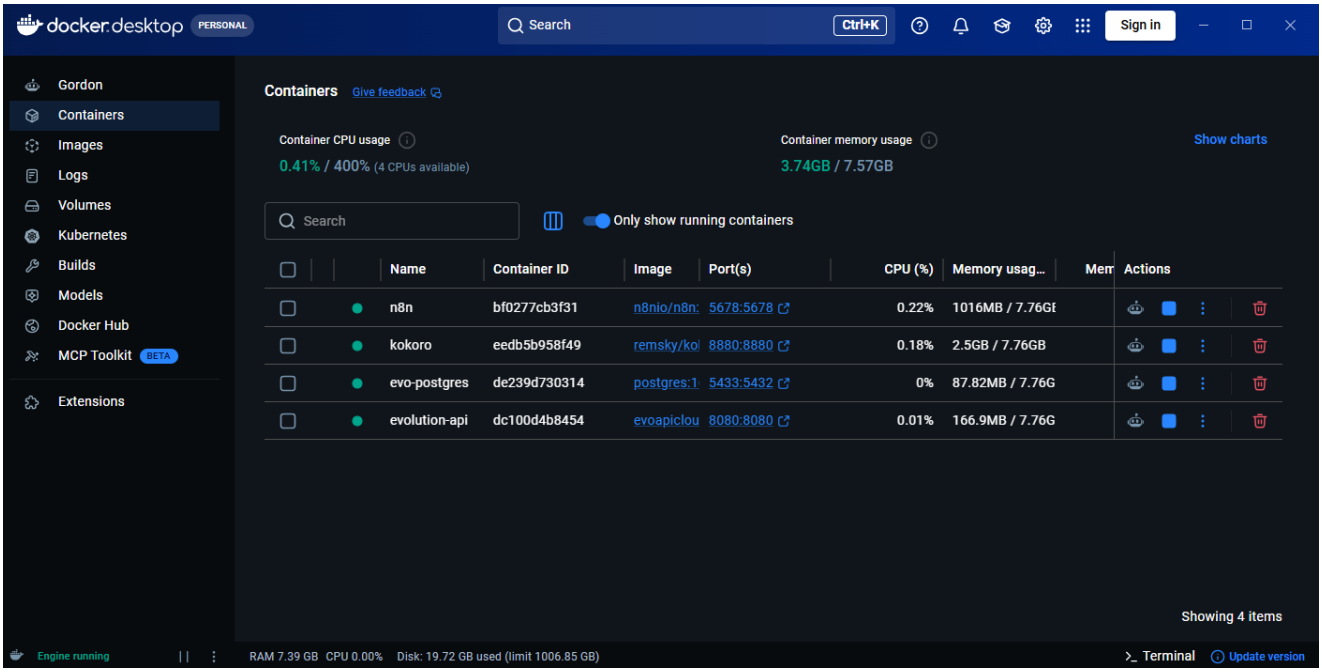


Fig. 3 — Docker Desktop: n8n (:5678), kokoro (:8880), evo-postgres (:5433) e evolution-api (:8080) em execução.

LLM local — Ollama · TTS pt-BR — Edge (Kokoro: fallback offline)

Ollama — LLM local (llama3.2:1b)

```
PS> Invoke-RestMethod http://127.0.0.1:11434/api/tags
PS> ollama list   # equivalente
ok=True  detail=HTTP 200
model_configured = llama3.2:1b
models disponíveis:
  - llama3.2:1b

Uso no pipeline: gera summary em linguagem natural a partir de metrics + alerts.
LLM local — sem enviar dados para API externa (LGPD / IA responsável).
```

Fig. 4 — Modelo llama3.2:1b no Ollama local.

TTS — áudios gerados (Edge Antonio)

```
PS> Get-ChildItem .\data\results\audio\*.mp3 | Sort LastWriteTime -Desc
  PULSE-TESTE-EDGE-antonio.mp3                164.4 KB
  PULSE-TESTE-PRONUNCIA.mp3                   806.7 KB
  PULSE-TESTE-MP3.mp3                          53.3 KB
total: 3 arquivo(s) .mp3 — TTS (Edge Antonio; Kokoro fallback)
```

Fig. 5 — Áudios .mp3 gerados pelo TTS (Edge, voz Antonio pt-BR).

3.1 n8n — workflows no editor

Workflows importados e prontos para execução (UI do n8n). Etapa 1 orquestra o microsserviço; Etapa 2 entrega em WhatsApp/Telegram/e-mail.

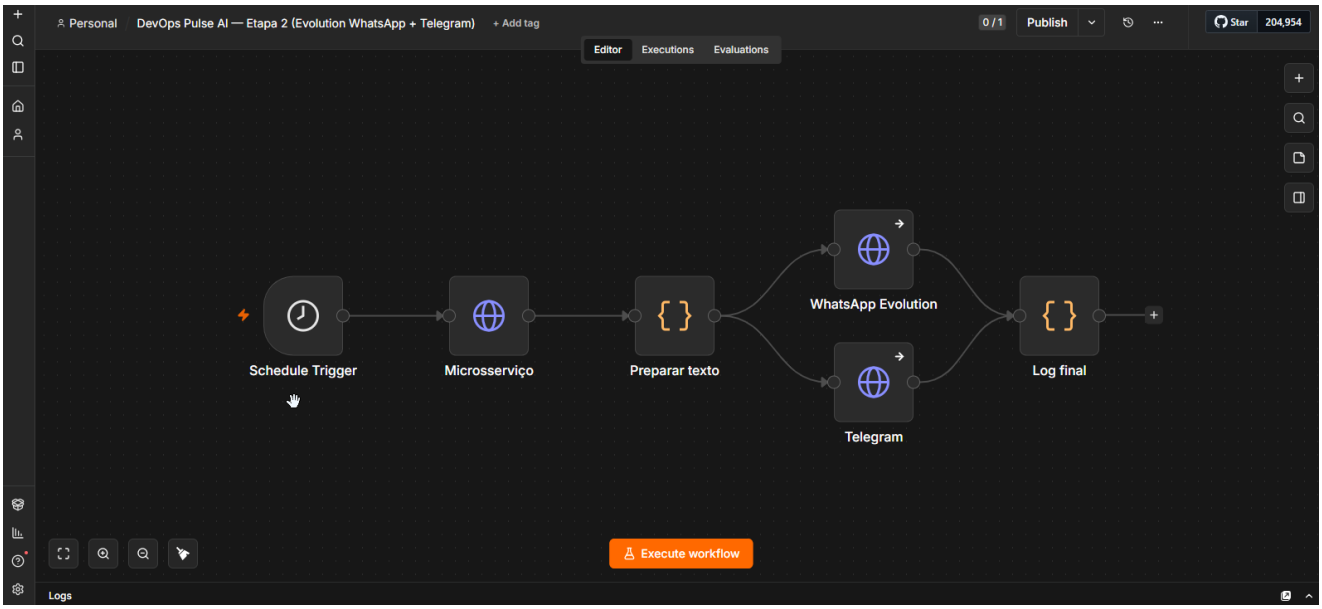


Fig. 6 — n8n UI · Etapa 2 (Evolution WhatsApp + Telegram): Schedule → Microsserviço → Preparar texto → WhatsApp/Telegram → Log final.

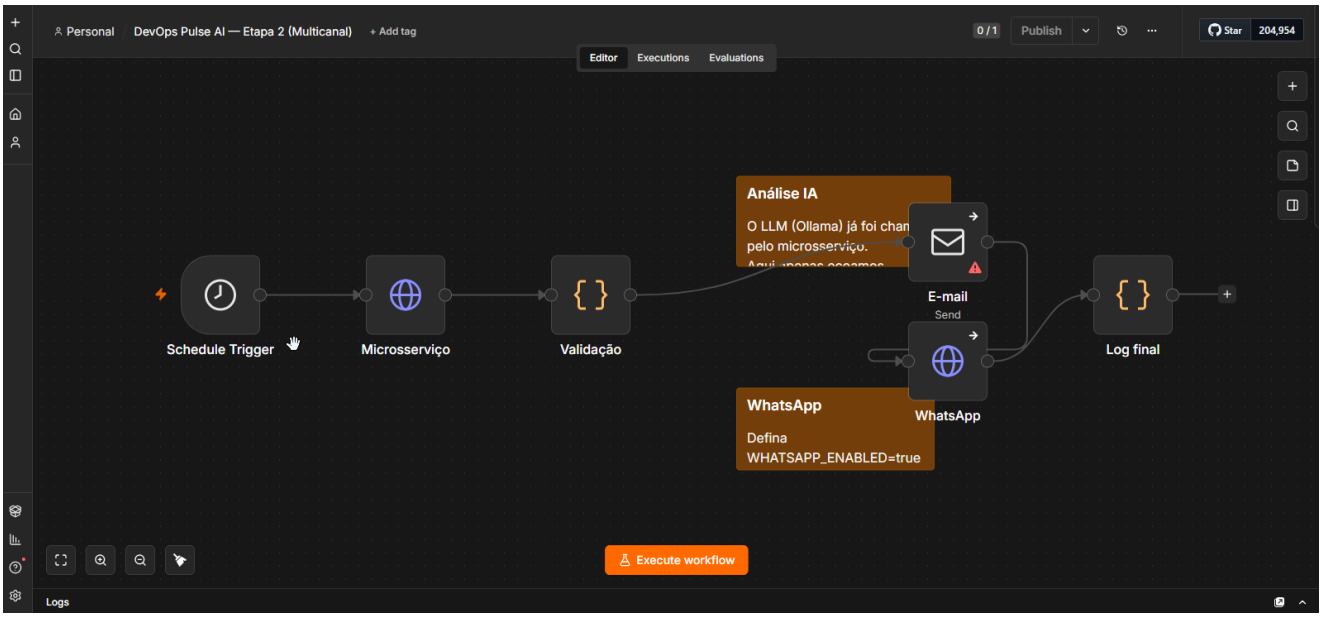


Fig. 7 — n8n UI · Etapa 2 (Multicanal): Schedule → Microsserviço → Validação → E-mail/WhatsApp → Log final.

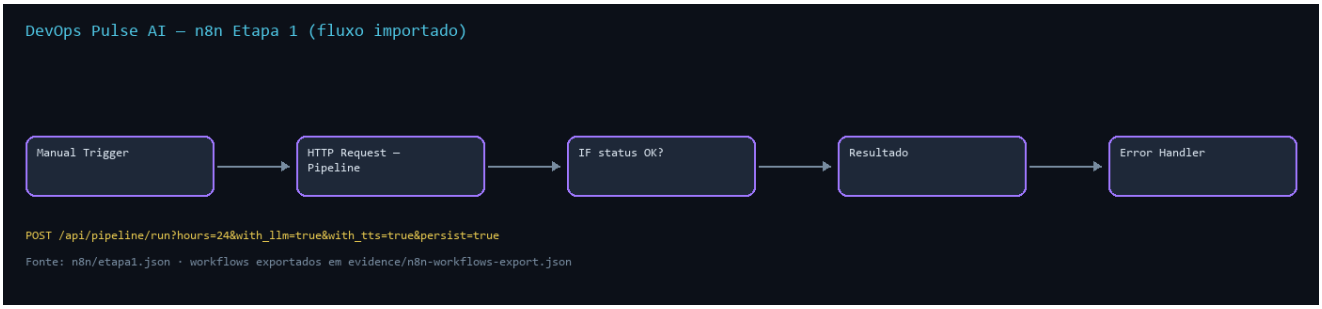


Fig. 8 — Diagrama da Etapa 1: Manual Trigger → HTTP pipeline → IF success.

```
n8n — workflows DevOps Pulse (CLI + API)
PS> docker exec -u node n8n n8n list:workflow
3En6gSalpfhghyM9 | Teste fastapi
8ReA2Ty1pEiKdUAF | JJBoleto - Chat IA - Ollama (hardened)
wOLIY6YnqeEo8asC | DevOps Pulse AI — Etapa 1
81YWZzdYxP7upBL2 | DevOps Pulse AI — Etapa 2 (Multicanal)
devops-pulse-etapa2-evo-tg | DevOps Pulse AI — Etapa 2 (Evolution WhatsApp + Telegram)

PS> docker exec -u node n8n n8n export:workflow --all  # 5 workflows exported
export file: evidence/n8n-workflows-export.json (27364 bytes)

PS> Invoke-RestMethod http://127.0.0.1:5678/api/v1/workflows -Headers @{X-N8N-API-KEY=...}
API publica respondeu 5 workflows (Etapa 1 + Etapa 2 ativas no n8n)
Observacao: UI do n8n exige login do owner (jone.cunha@gmail.com) — captura de tela da UI requer
senha local.
Evidencia de integracao: workflows importados + API key + export JSON.
```

Fig. 9 — Workflows DevOps Pulse listados no n8n (CLI + API).

4. Protótipo funcional (ID 2.2)

Consome API pública real (GitHub), gera resumo LLM (Ollama), sintetiza áudio TTS (Edge) e grava o resultado em banco simples (SQLite) com rastreio por execution_id.

POST /api/pipeline/run?hours=24&with_llm=true&with_tts=true&persist=true

Execução de referência

Campo	Valor
execution_id	PULSE-20260917-121306-7999
Status / fonte	success / github (n8n-io/n8n)
Latência total	82 549 ms (coleta 5,5 s · LLM 58,5 s · TTS 18,6 s · persist 11 ms)
Saídas	JSON + summary + audio .wav + linha SQLite

API /api/pulse — métricas reais (terminal)

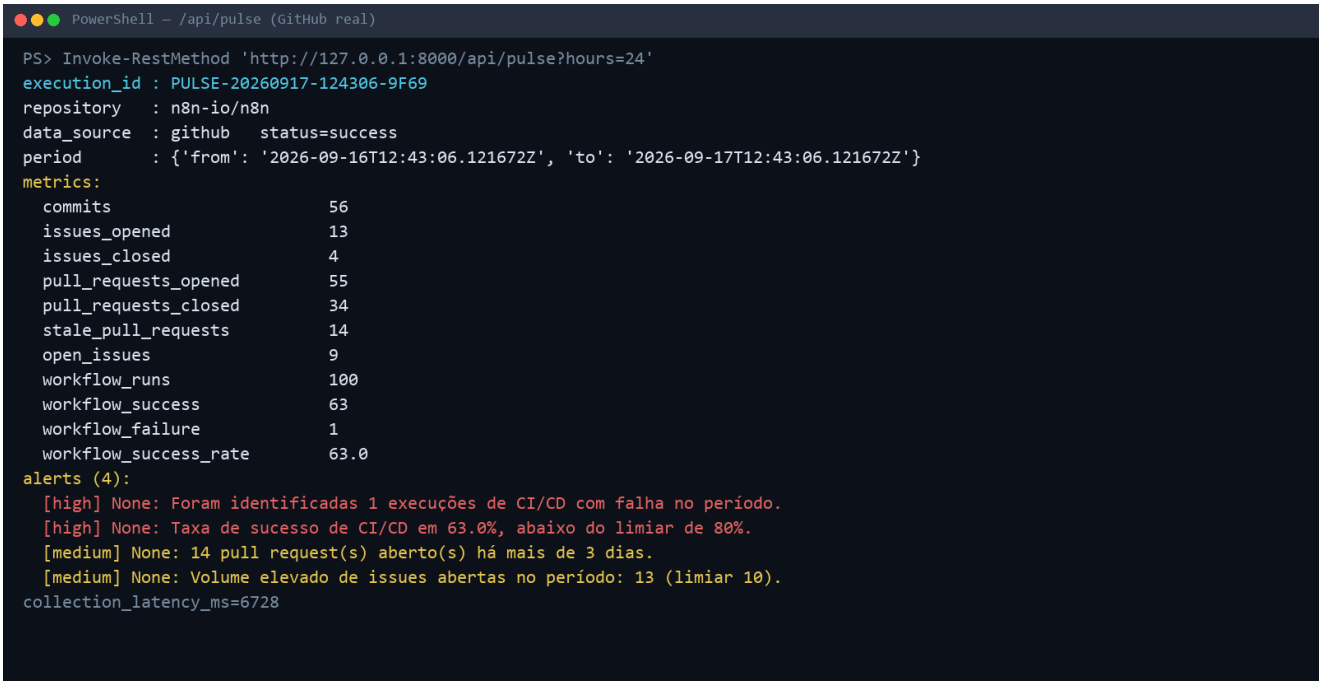


Fig. 10 — Payload com metrics, alerts, timings e data_source=github.

Logs estruturados do pipeline

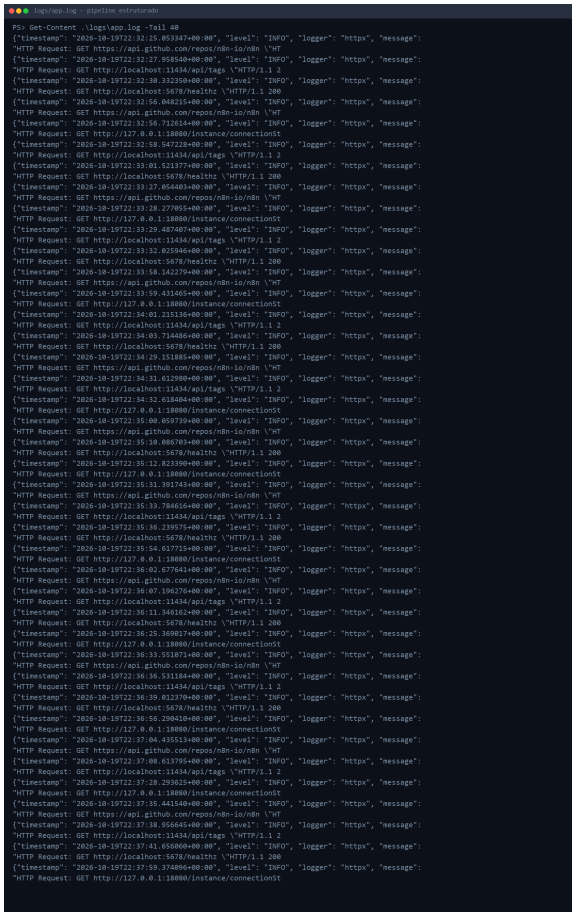


Fig. 11 — logs/app.log com etapas e execution_id.

4.1 Persistência — SQLite (grid para apresentação)

No lugar de abrir um SGDB, a página dashboard/sqlite.html exibe o histórico com badge da fonte dos dados (github vs demo).

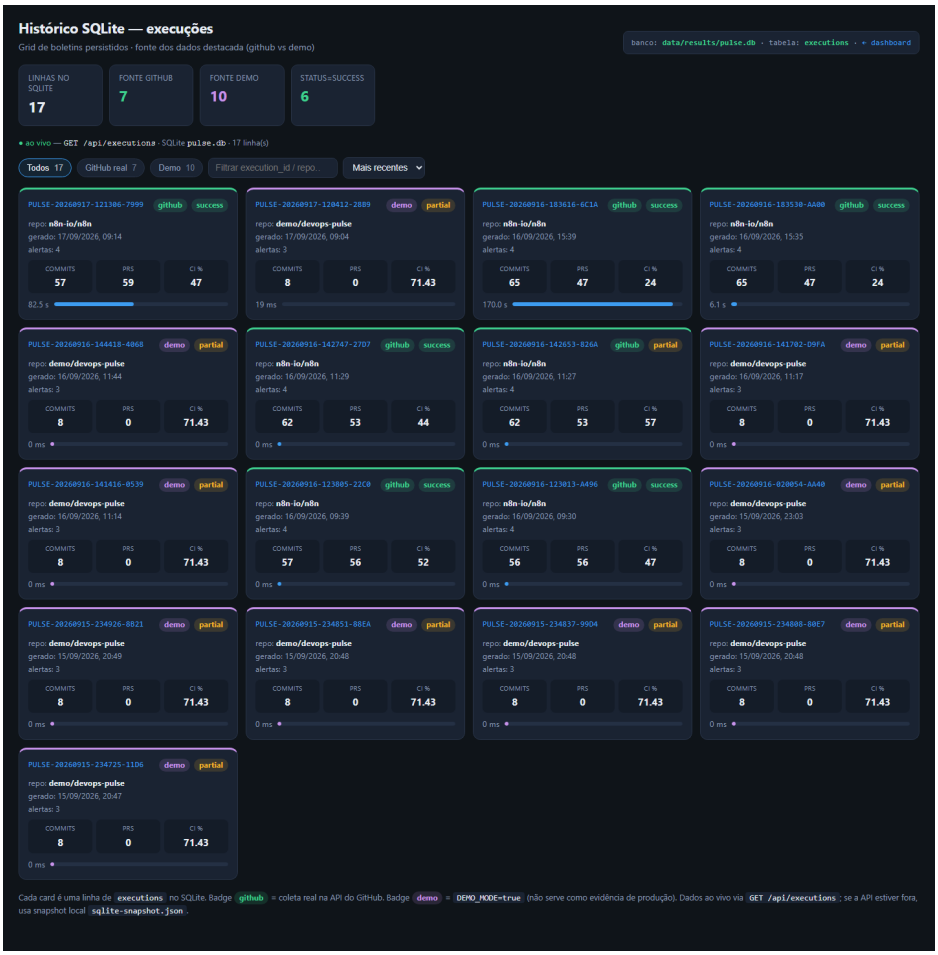


Fig. 12 — Grid de execuções do SQLite com fonte, status, métricas e latência.

```
SQLite — data/results/pulse.db
PS> sqlite3 data\results\pulse.db  # via python
tables: executions, sqlite_sequence
  executions: 72 rows
  sqlite_sequence: 1 rows
-- latest from executions --
{'id': 72, 'execution_id': 'PULSE-20261019-200955-8ACB', 'generated_at':
'2026-10-19T20:09:55.115708+00:00', 'repository': 'demo/devops-pulse', 'status': 'partial',
'data_source': 'demo', 'metrics_json': '{"commits": 8,
{'id': 71, 'execution_id': 'PULSE-20261019-200955-ED25', 'generated_at':
'2026-10-19T20:09:55.002707+00:00', 'repository': 'demo/devops-pulse', 'status': 'partial',
'data_source': 'demo', 'metrics_json': '{"commits": 8,
{'id': 70, 'execution_id': 'PULSE-20261019-200936-D48C', 'generated_at':
'2026-10-19T20:09:36.800562+00:00', 'repository': 'demo/devops-pulse', 'status': 'partial',
'data_source': 'demo', 'metrics_json': '{"commits": 8,
{'id': 69, 'execution_id': 'PULSE-20261019-194524-2035', 'generated_at':
'2026-10-19T19:45:24.838802+00:00', 'repository': 'demo/devops-pulse', 'status': 'partial',
'data_source': 'demo', 'metrics_json': '{"commits": 8,
{'id': 68, 'execution_id': 'PULSE-20261019-194524-5DE7', 'generated_at':
'2026-10-19T19:45:24.777242+00:00', 'repository': 'demo/devops-pulse', 'status': 'partial',
'data_source': 'demo', 'metrics_json': '{"commits": 8,
```

Fig. 13 — Tabela executions no banco pulse.db.

4.2 Dashboard executivo

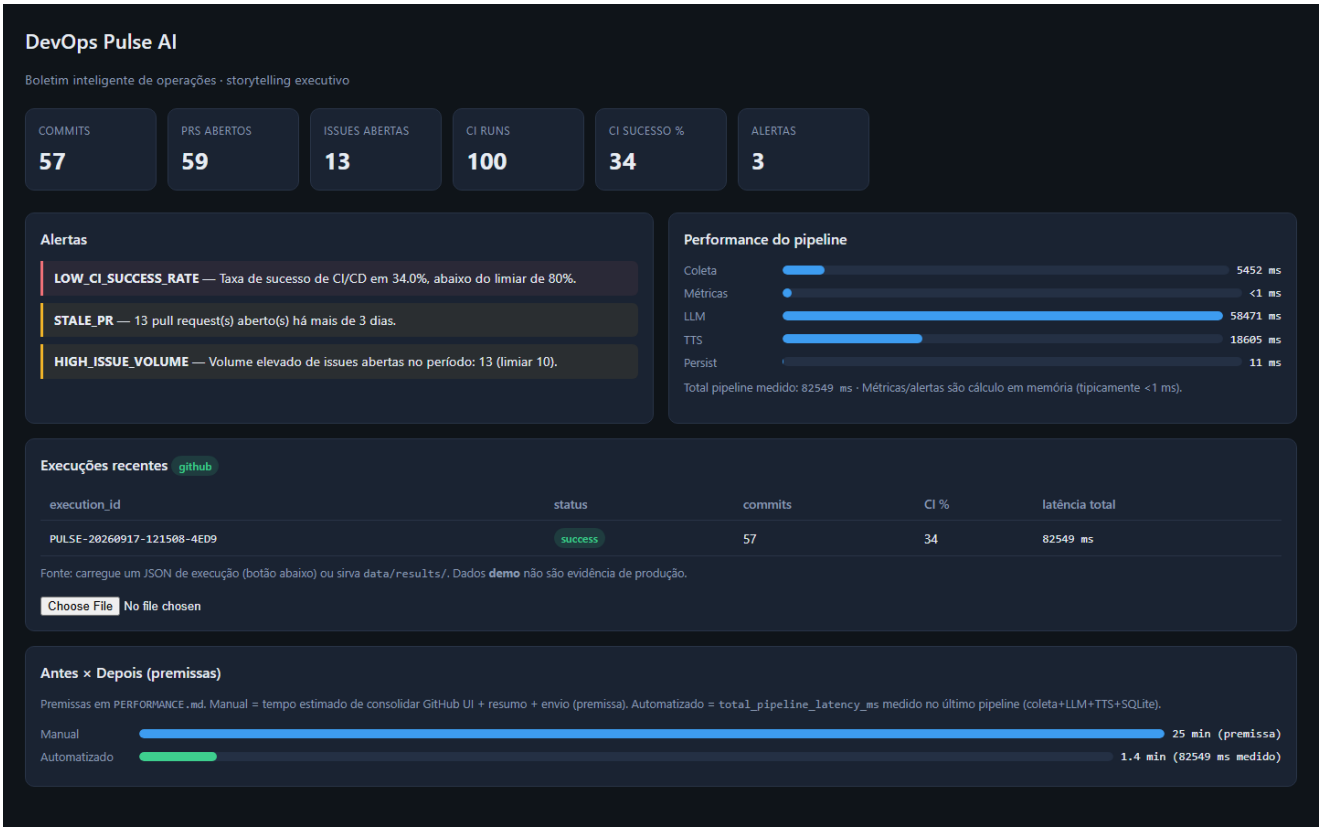


Fig. 14 — Cards, alertas, latências por etapa e antes × depois.

4.3 Robustez — testes e stress

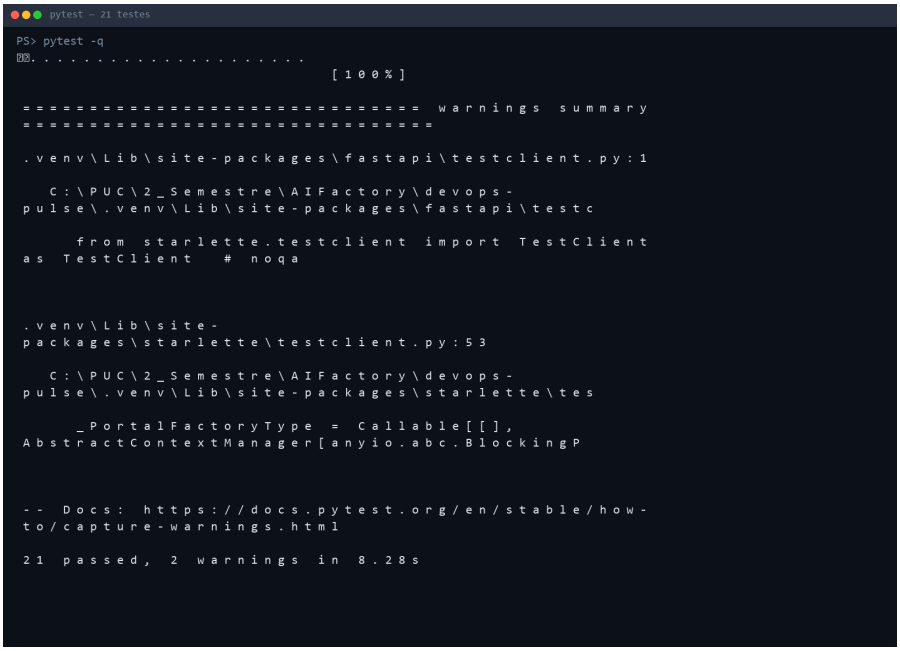


Fig. 15 — pytest: 21 passed.

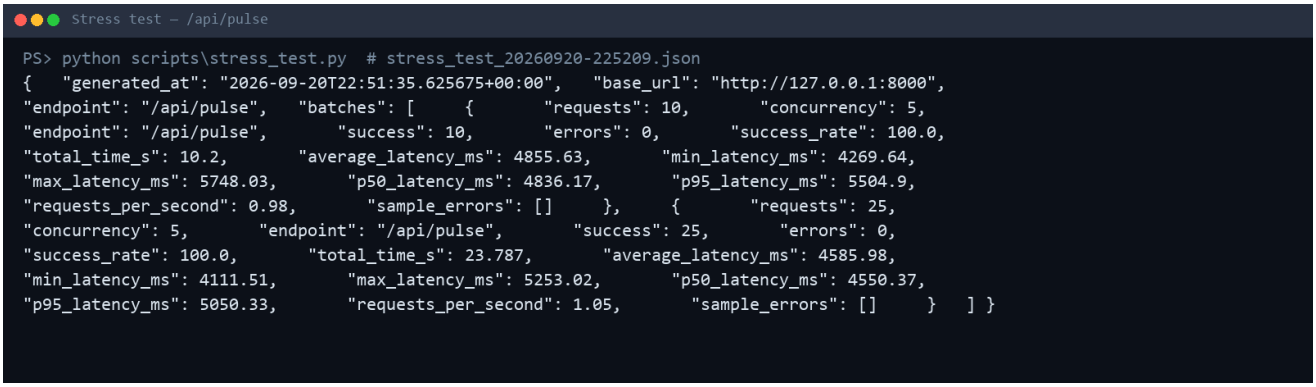


Fig. 16 — Stress test /api/pulse (10/25/50 req, 100% sucesso).

5. Entrega multicanal — WhatsApp (n8n + Evolution API)

Boletim completo enviado ao grupo DevOps Pulse AI no formato consolidado (indicadores + alertas + stress + pipeline).

[DevOps Pulse AI] Boletim - PULSE-20260920-201719-5A27

Repo: n8n-io/n8n
Fonte: github | Status: success

Indicadores (24h):
. Commits: 2
. PRs abertos: 8 | fechados: 5
. Issues abertas: 7
. CI success rate: 16.0%
. Latência coleta: 5086 ms

Alertas:
• [high] Foram identificadas 2 execuções de CI/CD com falha no período.
• [high] Taxa de sucesso de CI/CD em 16.0%, abaixo do limiar de 80%.
• [medium] 44 pull request(s) aberto(s) há mais de 3 dias.

Stress test (10/25/50 req): 100% sucesso • p95 ~6,6s
• Pipeline: GitHub → metrics → alerts → n8n

Comprovação de entrega no WhatsApp (grupo DevOps Pulse AI)

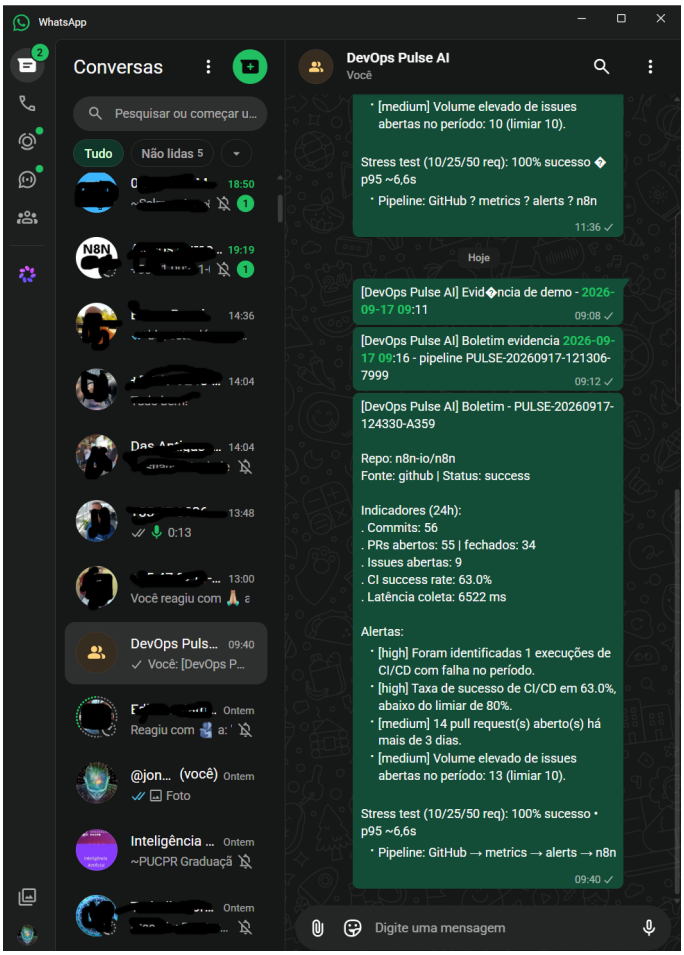


Fig. 17 — Conversa WhatsApp: boletim completo PULSE-20260917-124330-A359 (repo n8n-io/n8n · fonte github · CI 63% · alertas + stress + pipeline).



Fig. 18 — Comprovante de envio via Evolution API (key + remoteJid do grupo).

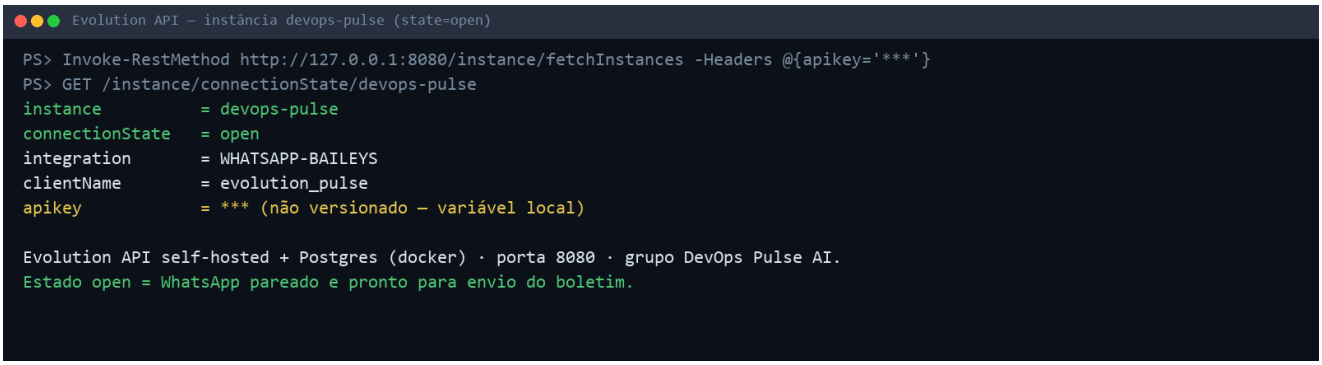


Fig. 19 — Evolution API / instância devops-pulse (connectionState=open · WhatsApp pareado).

6. Performance e responsabilidade (LGPD / IA)

Cenário	Tempo	Origem
Manual (premissa)	25 min	Consolidar GitHub UI + resumo + envio
Automatizado (medido)	82,5 s	PULSE-20260917-121306-7999
Redução	~94% no ciclo completo	(25 − 1,4) / 25 com pipeline de ~1,4 min

Princípios aplicados

- O LLM não é fonte primária — só interpreta métricas já calculadas (reduz alucinação).
- Minimização: Ollama recebe apenas métricas + alertas.
- Tokens e chaves só em .env / pulse.env (gitignored).
- Modo demo rotulado — nunca apresentar demo como produção.
- Fallbacks explícitos: LLM_ERROR, TTS_ERROR, PERSISTENCE_ERROR, onError continue no n8n.

7. Aprendizagem-chave — input → process → output

Etapa	O que acontece	Tecnologia
INPUT	Coleta de commits, PRs, issues e workflow runs	GitHub API + httpx + retry
PROCESS	Métricas, alertas, resumo em linguagem natural, síntese de voz	Python, Ollama, Edge TTS
OUTPUT	JSON, áudio, SQLite, dashboard e notificação	FastAPI, SQLite, n8n, Evolution

O protótipo demonstra que é possível conectar sistemas via API, transformar dados em indicadores acionáveis e automatizar a comunicação operacional — o primeiro passo da transformação digital do desafio real.

Como demonstrar ao vivo (roteiro curto)

- curl /health → componentes no ar (demo_mode=false)
- curl /api/pulse → métricas reais do n8n-io/n8n
- POST /api/pipeline/run → logs + áudio + SQLite
- Abrir dashboard/sqlite.html → grid com fonte github
- Mostrar WhatsApp do grupo com o boletim completo

8. Anexos — evidências adicionais

Actions/CI do repositório monitorado

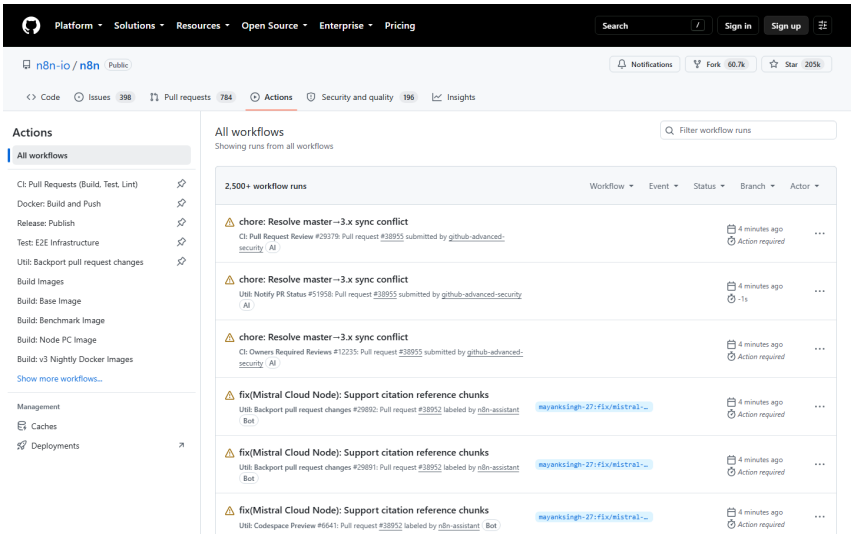


Fig. anexa — 13-github-actions.png

Pull requests abertos (fila manual hoje)

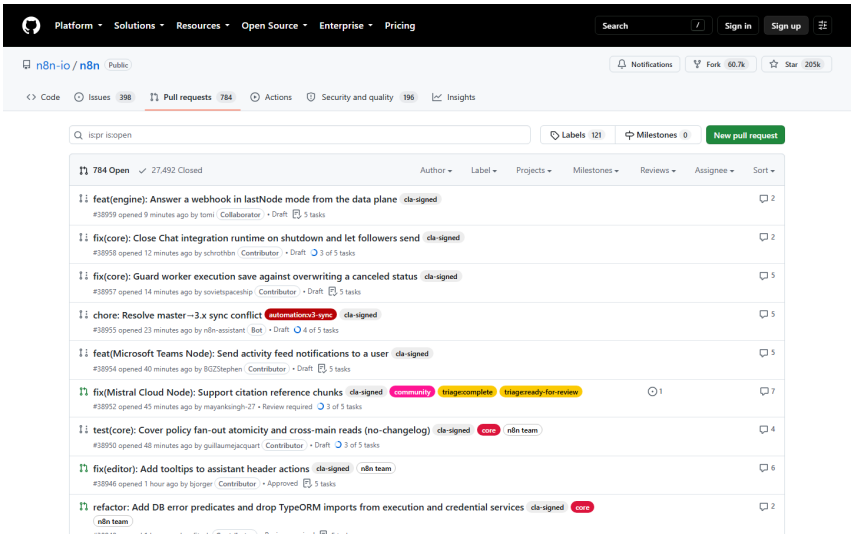


Fig. anexa — 13-github-pulls.png

GET /health — componentes do sistema (terminal)

```
GET /health — componentes do DevOps Pulse AI

PS> Invoke-RestMethod http://127.0.0.1:8000/health
status     = ok
demo_mode  = False  # false = dados reais do GitHub
components:
  [OK  ] github     HTTP 200
  [OK  ] ollama     HTTP 200  model=llama3.2:1b
  [OK  ] kokoro     /health HTTP 200
  [OK  ] n8n        HTTP 200

4 componentes monitorados: GitHub API · Ollama · Kokoro · n8n.
Se um cair, o pipeline degrada com fallback rotulado — não trava o boletim.
```

Fig. anexa — 01-api-health.png

n8n UI — workflow de teste FastAPI (Webhook → HTTP Request)

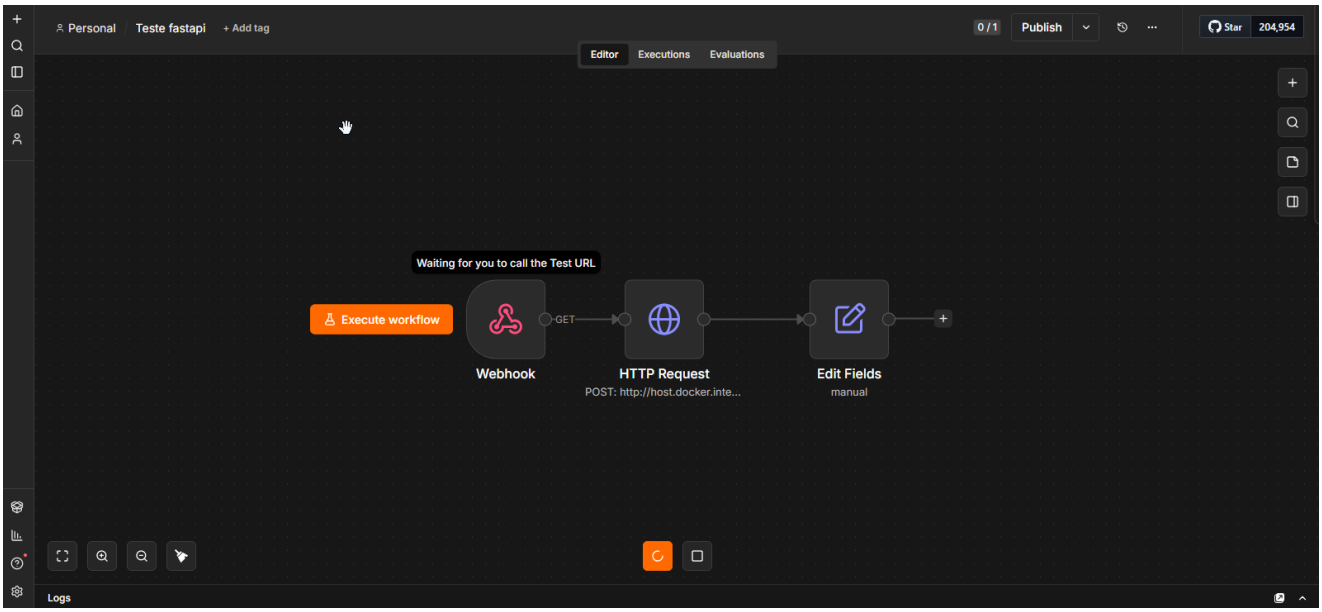


Fig. anexa — n8n_3.png

UI do n8n protegida (login do owner)

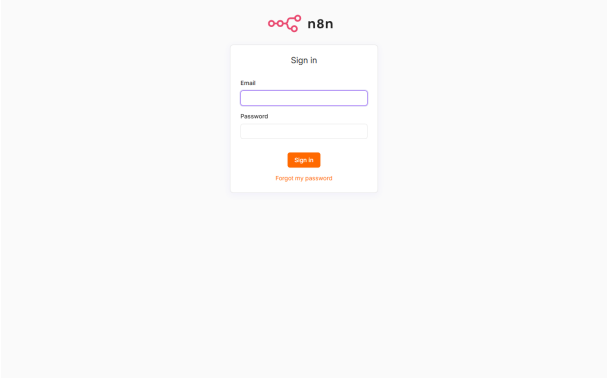


Fig. anexa — 03-n8n-login.png

Canal de e-mail (reserva) documentado



Fig. anexa — 08-email.png

Arquivos do projeto: C:\PUC\2_Semestre\AIFactory\devops-pulse
Documentação: README.md · ARCHITECTURE.md · docs/diagnostico.md · docs/canvas.md · ETHICS-LGPD.md · PERFORMANCE.md